Leases for Job Queues in Postgres

A. Made-up and B. Example
Example University

Abstract

We describe a job queue that keeps its jobs in one Postgres table. Leases make it safe: a job that loses its worker comes back after its lease ends.

1 Introduction

A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue.

Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop.

A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock.

When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once.

This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error.

2 Leases

When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once.

This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error.

The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row.

We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends.

A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs.

3 Heartbeats and the reaper

We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends.

A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time

when the lease ends. A heartbeat moves the end of the lease forward while the job still runs.

A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue.

Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop.

A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock.

4 Results

Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop.

A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock.

When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once.

This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error.

The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row.

The code is at https://example.com/queue-notes and the data is on our data page.

5 Related work

This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error.

The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row.

We measured 3,000 jobs with four workers, and each job ran exactly once. A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends.

A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs.

A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue.

6 Conclusion

A job that fails three times goes to a dead state, where a person can read its last error. A worker claims a job

with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs.

A worker claims a job with SELECT ... FOR UPDATE SKIP LOCKED, so two workers never take the same row. Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue.

Each claim writes a lease: the name of the worker and the time when the lease ends. A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop.

A heartbeat moves the end of the lease forward while the job still runs. When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock.

When a worker stops without a word, its lease ends, and a reaper gives the job back to the queue. This design is efficient for a small team, and it works offline on one laptop. The workflow needs no broker: the table is the queue, and a transaction is the only lock. We measured 3,000 jobs with four workers, and each job ran exactly once.

References

[1] C. Made-up. Queues in practice. 2025.